Communication

PostHog handbook, contents/handbook/company/communication.md at commit f9d5711. Copyright (c) 2020-2025 PostHog Inc., MIT-style license for /contents (see LICENSE-posthog-contents.txt). Text unchanged; rendered to PDF for the Kohu demo. PostHog does not endorse Kohu.

Introduction

With team members across many countries, it's important for us to practice clear communication in ways that help us stay connected and work more efficiently.

To accomplish this, we use **asynchronous communication as a starting point** and stay as open and transparent as we can by communicating on GitHub through public issues and pull requests, as well as in our PostHog User and internal Slack.

Our communication values

1. **Assume positive intent.** Always coming from a position of positivity and grace.
2. **Form an opinion.** We live in different locations and often have very different perspectives. We want to know your thoughts, opinions, and feelings on things.
3. **Feedback is essential.** Help everyone up their game in a direct but constructive way.

Golden rules

1. Use **asynchronous communication** when possible: pull requests (preferred) or issues. Announcements happen on the appropriate Slack channels and people should be able to do their work without getting interrupted by chat.
2. Discussion in GitHub issues or pull requests is preferred over everything else. If you need a response urgently, you can Slack someone with a link to your comment on an issue or pull request, asking them to respond there. However, be aware that they still may not see it straight away (and that's OK in our book). That said, casual conversations in Slack are completely normal — it’s our main space for day-to-day communication.
3. You are not expected to be available all the time. There is **no** expectation to respond to messages outside of your planned working hours.
4. It is 100% OK to ask as many questions as you have - please ask in public channels! If someone sends you a handbook link, that means they are proud that we have the answer documented - they don't mean that you should have found that yourself or that this is the complete answer. If the answer to a question isn't documented yet please immediately make a pull request to add it to the handbook in a place you have looked for it.
5. When someone asks for something, reply back with a deadline or by noting that you already did it. Answers like: 'will do', 'OK', or 'it is on my todo list' are not helpful. If it is a small task for you but will unblock someone else, consider spending a few minutes to do the task so the other person can move forward.
6. By default, avoid creating private groups for internal discussions.

Public by default

We make things public by default because transparency is core to our culture. The kinds of information we share falls into one of three buckets:

- *Public* - most things, including our product, roadmap, handbook and strategy.
- *Shared internally* - almost everything else, such as financial performance, security, fundraising and recruitment.
- *Private internally* - personal team information, i.e. compensation, disciplinary issues.

Information that is not publicly shared is in areas with complex signals that can impact our ability to sell, raise money or are inappropriate to share more widely for personal privacy reasons.

We have two repos to centralize and document private internal communication. These are the source of truth for any internal information, and anything that should be written down (as established in these guidelines) should live in these repos or (better) in this Handbook, not on Slack. This will make it easier when having to search for older stuff, sharing context between public and internal repos, and for newcomers to have all information they might need readily available.

Company Internal

Repository can be found in https://github.com/PostHog/company-internal

Documents any company-wide information that can't be shared publicly within People, Ops, Legal, Finance or Strategy.

Examples of information that should go here:

- ✅ Hiring plans and discussions *before* we post a job ad
- ✅ People discussions, e.g. benefits, pensions, share options, org structure
- ✅ Onboarding/offboarding checklists
- ✅ Non-engineering team sprint planning (as these will often be a mix of public and private tasks and we don't want to restrict people)
- ✅ Sensitive discussions about future positioning, customer strategy, fundraising, board meetings
- ✅ [Sometimes] Discussions about replacing or adding tools, services, and systems that we use

For company-related issues that *can* be discussed publicly, these should go in the `meta` repo which can be found in https://github.com/PostHog/meta/

Examples of information that should NOT go here:

- ❌ Any information that should be public (see guidelines on public by default), this should go in the public repositories (`posthog`, `posthog.com`, `meta`, ...). Things like:
  - Some marketing campaigns where it doesn't matter if our competitors see it; retros after campaigns
  - Offsite planning and retros
  - Discussions about future positioning and strategy that will end up in the Handbook anyway
  - Discussions about tools where there isn't a security risk and it interfaces with our customers (e.g. marketing, customer support)

    - Generally anything that will end up in the Handbook anyway, including culture and values discussions
- ❌ Bug reports, security issues, or any other engineering-related discussions. These should go in the Product Internal repo.
- ❌ Billing issues, product or growth discussions. These should go in the Product Internal repo.

Product Internal

Repository can be found in https://github.com/PostHog/product-internal

Contains internal information related to the PostHog product. Documents any non-public information (as established in these guidelines) that specifically relates to engineering, product, growth or design.

This repository was introduced to aid maintenance and day-to-day usage of internal repositories. Having these discussions together with the company-wide information proved unwieldy. More context on this decision.

> Please be sure to read the README of the repo for guidelines on how to file specific issues.

Examples of information that should go here:

- ✅ Vulnerabilities (security bugs) reports
- ✅ Bug reports where most of the context of the report depends on customer's PII. *Some bug reports require screenshots, recordings, or some other information that contains PII and as such can't be public.*
- ✅ Post-mortems on outages, or other issues affecting a large portion of customers. The results of these should usually be made public though.
- ✅ Documentation of internal infrastructure, where if it was public knowledge could provide valuable information to an attacker.
- ✅ Experiment (A/B testing) results.
- ✅ Product or growth strategy discussions (unless they should be public).
- ✅ Interview exercises or questions for engineering, product, growth or design tasks that should not be public.
- ✅ Documentation of engineering or product requirements documents that can't be public (these should be quite rare).
- ✅ Billing or pricing-related discussions that is not yet public.

Examples of information that should NOT go here:

- ❌ Any information that should be public (see guidelines on public by default), this should go in the public repositories (`posthog`, `posthog.com`, `meta`, ...).
- ❌ Any internal information that does not fall under the scope of purely engineering, product, growth or design. This should go in the Company Internal repo if private or `meta` if public.
- ❌ Bug reports that don't contain any PII or where the PII only contains supporting information. In this case, file the bug under the relevant public repo and add a protected link to the additional information (e.g. a private Slack link, or a link to this repo).

Written communication

GitHub

Everything starts with a pull request

It's best practice to start a discussion where possible with a Pull Request (PR) instead of an issue. A PR is associated with a specific change that is proposed and transparent for everyone to review and openly discuss. The nature of PRs facilitate discussions around a proposed solution to a problem that is actionable. A PR is actionable, while an issue will inevitably lead to a longer period before the problem is addressed.

Always open a PR for things you are suggesting and/or proposing. Whether something is not working right or we are iterating on new internal process, it is worth opening a pull request with the minimal viable change instead of opening an issue encouraging open feedback on the problem without proposing any specific change directly. Remember, a PR also invites discussion, but it's specific to the proposed change, which facilitates focused decisions.

By default, pull requests are **non-confidential**. However, for things that are not public please open a confidential issue with suggestions to specific changes that you are proposing. When possible, consider not including sensitive information so the wider community can contribute.

Not every solution will solve the problem at hand. Keep discussions focused by *defining the problem first* and *explaining your rationale* behind the Minimal Viable Change (MVC) proposed in the PR. Have a bias for action and don't aim for consensus - some improvement is better than none.

Issues

GitHub Issues are useful when there isn't a specific code or document change that is being proposed or needed. For example, you may want to start an issue for tracking progress or for project management purposes that do not pertain to code commits. This can be particularly useful when tracking team tasks and creating issue boards.

However, it is still important to maintain focus when opening issues by defining a single specific topic of discussion as well as defining the desired outcome that would result in the resolution of the issue. The point is to not keep issues open-ended and to prevent issues from going stale due to lack of resolution. For example, a team member may open an issue to track the progress of a blog post with associated to-do items that need to be completed by a certain date (e.g. first draft, peer review, publish). Once the specific items are completed, the issue can successfully be closed.

> **Note:** If you're new to using GitHub, check out this handy primer - it's specific to how we use GitHub at PostHog. You'll learn the key concepts and how to manage notifications. It's important, as this is where the bulk of our company-wide communication happens. (Think of GitHub notifications as a replacement for your work email.)

Keeping on top of reviews, issues and notifications

Keeping track of everything that's happening in GitHub can be daunting, but it's important to make sure your team receives reviews and feedback on a timely manner.

To keep on top of this, we suggest going through issues where you've been mentioned regularly. Some tricks which can help are:

- (Highly recommended) Turn on Github slack notifications
    - This will send you a slack notification when someone mentions you in a PR or issue. You can also get periodic reminders for PRs that you've been requested to review.
- (Highly recommended) Join the `#github-rfcs` channel on Slack. This is where we post all the RFCs.
- Turning on GitHub email notifications and use filters to focus on your teams activity.
- Using the GitHub notifier extension.

Tip for easy searching through everything

To search all code, PRs and issues ever written at PostHog you can search everything in the PostHog organization on Github. To do that can go to github.com/posthog and search in the top left corner.

For extra convenience, you can also add this search as a 'search engine' in Chrome. That way you can type in `ph <tab>` and instantly find anything. To do that, follow these steps:

1. Hit `command + ,` in your browser
2. Type `search`, find "manage search engines"
3. Click "add" next to "other search engines"
4. For "Search engine" type in `github posthog organization`
5. For "keyword" type in `ph`
6. For "url" copy in `https://github.com/search?q=org%3Aposthog+%s&type=issues`

You can now type `ph + tab` into your browser and search issues directly

Slack

Slack is used for more informal communication, or where it doesn't make sense to create an issue or pull request. Use your judgment to determine the appropriate channel, and whether you should be chatting publicly (default) or privately.

Also keep in mind that, as an open source platform, PostHog has contributors who don't have access to Slack. Having too much context in a private location can be detrimental to those who are trying to understand the rationale for a certain decision.

> Slack canvasses are useful for storing information like schedules, bookmarks, personal to-do lists, scratch notes etc. However, things like quarterly goals, runbooks, sprint plans, FAQs etc. should live in the Handbook, Docs, or in a GitHub RFC or Issue by default. If you find yourself documenting something useful in Slack, it's much better to put it in GitHub instead and link to it from Slack so that PostHog AI can include it in future search results. Slack canvasses are terrible for searchability!
>
> Slack recap is a great way to learn from others by adding channels like `#ask-max` and `#today-i-learned` to the recap. You can also use it to keep tabs on teams you may not directly work on, but still want to know what's being discussed.

Slackbot is a handy AI agent that can search across the PostHog workspace in Slack to help answer your questions. If you're looking for information or trying to find a past conversation, Slackbot is a great place to start.

Keeping up with what we ship

We recommend everyone at PostHog joins `#changelog` to stay current on what's shipped. It's owned by the Wizard & Docs team and updates constantly as PRs merge.

The channel is powered by agentic workflows that scan merged PRs and feature flag changes and summarize them into it. PRs that are still behind a feature flag stay out, based on the *Release status* section of the `posthog/posthog` PR template. See how to publish changelog for more detail.

Slack etiquette

Slack is used differently in different organizations. Here are some guidelines for how we use Slack at PostHog:

1. Keep `#general` open for company-wide announcements.
2. `@channel` or `@here` mentions should be reserved for urgent or time-sensitive posts that require immediate attention by everyone in the channel. (Examples: changing a meeting invite URL just before a meeting, or soliciting urgent help for a service disruption, where you're not sure who is immediately available)
3. Make use of threads when responding to a post. This allows informal discussion to take place without notifications being sent to everyone in the channel on every reply.
4. When possible, summarize multiple thoughts into a single message instead of sending multiple messages sequentially.
5. You don't need to tell people if you're away from your computer, especially on no-meeting days. There's no general expectation people are available to reply to messages in real time, including in Slack.
6. Keep your Slack profile up to date with the right information, including the appropriate name eg with surname or surname initial if you share a name with a colleauge.

Channel naming conventions so people don't get confused:

- `#team-[team-name]` - small team channels, only as listed on the teams page
- `#project-[project-name]` - one-off initiatives that may involve people across multiple teams, but don't fit neatly into a team channel
- `#posthog-[customer-name]` - shared channels with *customers* only (if you want to create a shared channel with an external partner, use `#[partner-name]-posthog` instead)
- `#alerts-[team-name]` - useful to create a separate channel for your team to send alerts into, so your main channel doesn't get noisy
- `#support-[product-name]` - similarly, useful to feed support requests in if helpful without adding clutter
- `#offsite-[team]-[month]-[year]-[where]` - For planning and coordination of team offsite events
- `#onboarding-[who]-[team]-[month]-[year]-[where]` - For coordinating and supporting new team member onboarding
- `#hiring-[team-name]` - For recruiting discussions, candidate feedback, and hiring coordination for a specific team

- `#superday-[first-name]-[role]` - For candidate interview coordination and feedback during intensive interview days

On the very rare occasions you need to create a private channel for some reason - most commonly hiring-related - then it's probably worth sticking `#private-xxxxx` in front so people don't accidentally add external parties who shouldn't be in there.

Google Docs and Slides

Never use a Google Doc / Slides for something non-confidential that has to end up on the website or this handbook. Work on these edits via commits to a pull request. Then link to the pull request or diff to present the change to people. This prevents a duplication of effort and/or an out of date handbook.

We mainly use Google Docs to capture internal information like meeting notes or to share company updates and metrics. We always make the doc accessible so you can comment and ask questions.

Please avoid using presentations for internal use. They are a poor substitute for a discussion on an issue. They lack the depth, and don't add enough context to enable asynchronous work.

When giving a talk which requires a presentation, use Pitch to build your slides. (It offers more control over design than Google Slides.) They also have a desktop app. We don't (yet) have templates configured, but you can draw from existing slides in other presentations - just copy/paste into your own presentation and modify accordingly. If you'd like assistance with slide design (or using Pitch), ask in the `#team-graphics` Slack channel.

James (H) is an admin on the Pitch account. Because Pitch charges per seat, we remove users who only need periodic access but can easily re-add when needed.

Email

1. Internal email should be avoided in nearly all cases. Use GitHub for feature / product discussion, use Slack if you cannot use GitHub, and use Google Docs for anything else.
2. The only uses we have for internal email are:
    - Obtaining approvals for legal things
    - Sending some types of more official company documents (e.g. job offers, payroll forms)
    - Communicating with external partners

Writing style

1. We use American English as the standard written language in our public-facing comms, including this handbook. This extends to date formats (September 4, 2021) and defaulting pricing to the US Dollar ($42).
2. Do not use acronyms when you can avoid them. Acronyms have the effect of excluding people from the conversation if they are not familiar with a particular term.
3. Common terms can be abbreviated without periods unless absolutely necessary, as it's more friendly to read on a screen. (Ex: *USA* instead of *U.S.A.*, or *vs* over *vs.*)
4. We use the Oxford comma.

5. Do not create links like "here" or "click here". All links should have relevant anchor text that describes what they link to. Using meaningful links is important to both search engine crawlers (SEO) and people with accessibility issues.
6. We use sentence case for titles.
7. When writing numbers in the thousands to the billions, it's acceptable to abbreviate them (like 10M or 100B - capital letter, no space). If you write out the full number, use commas (like 15,000,000).

Requests for comment (RFCs)

We use RFCs to communicate and gather feedback on a decision. RFCs are useful because they help us stay transparent, and the process of writing them forces you to clearly articulate your thoughts in a structured way.

Here are the steps for an RFC:

1. Identify a problem and a decision to be made
2. Create an RFC as a pull request using one of the RFC templates.
    - Using a template isn't a requirement, though it is a helpful and recommended starting place if you haven't written many RFCs here before. You can also get inspiration from other RFCs, as many have different sections and styles depending on the type of thing being discussed.
3. Share the RFC:
    - Assign people whom this RFC will impact, or who may have good opinions on the topic, as reviews to the pull request.
    - Post in the relevant Slack channel (normally the teams slack channel, or `#tell-posthog-anything` if it's a bigger cross-team RFC)
    - The aim is not to get one person to mark the RFC as approved, but to get the right people involved and commenting. It's a request for *comments* not a request for *approval*. That means you might need to chase them to make sure it happens - tag the people, present at all hands, use offsites or other sync time - **it's your responsibility to nudge the relevant people for their input**
4. If an RFC is cross-team and is causing a large amount of disagreement, it might be worth having a sync meeting to reach a decision
5. Once a decision is made, include the decision in the pull request, merge it in and share this in the relevant channel and `#github-rfcs` again.

When does it work best to write an RFC?

Writing an RFC may be helpful when any of the following is true:

- You want to clarify something for yourself or it affects just one team
- It is a relatively non controversial change/idea that doesn't require much extra context. In other words, it doesn't create problems for another team.
- It will be a large amount of work (more than 2-3 weeks of people's time)
- It's introducing a new technology
- It's a major new feature, change to the product, or change to the company
- It will have a major customer impact

When does a meeting / another approach work better than an RFC?

An RFC is likely to be unhelpful as a first step in other circumstances. Specifically, when you want to ship or suggest a change to something that significantly affects teams outside your own. In this instance, we've seen that RFCS can lead to 10 to 25+ comments, which feels antagonistic (teams having to explain all the context around their strategy down to why this decision is something they perhaps disagree with), and creates a lot of work. A single call in this instance is likely much faster than lots of frustrated people in 1/1s talking about it *and* the energy/time needed to respond to everything in a long thread.

However please write notes on such a call - to ensure everyone *is* on the same page. This could then be copy pasted into an RFC for transparency's sake / future reference.

How should I use AI when writing RFCs?

RFCs are the place where we do concentrated, original thinking. This thinking should not be outsourced to AI. Even when given context, AI writes content that is average and unoriginal, difficult to read (adjective-stuffing, anyone?), and places emphasis on the wrong things. RFCs should be largely hand-written, especially sections that define the problem, potential solutions, and question/answer.

AI is still useful in the RFC process, however. Feel free to use AI for research and final draft polish (but please don't have it rewrite the whole thing for you). Analysis and research done by AI can be included in the RFC at the bottom as appendixes.

Top tips for RFCs

- RFCs can be very short and are often better than making decisions by Slack threads.
- You don't need to have a long decision-making time - 2 days is fine for smaller changes if you receive the relevant input and are confident in your decision.
- You don't need to reach full agreement to decide, particularly if the decision is reversible. Instead, it should be when the decision maker has considered the feedback and is confident in their decision.
- Double-check whether your input will be useful or add noise. Or wait for the people closest to first discuss the problem. At PostHog, we don't make decisions by committee - instead we have great people divide and conquer. This particularly applies to the controversial areas such as pricing.
- As the decision maker, you should use your judgment as to which comments you want to respond fully to. It's fine to politely decline a question if you think it's not required for the decision being made.
- If you're introducing new technologies, you'll likely want to tag someone from Team Infrastructure.
- You don't need to wait until the date you've said to make the decision if you've already consulted with the key people.
- Make it easy for others to give feedback, e.g. if you only need input from someone on the Infra team about adding websockets then say that, rather than leaving it for them to work out
- Write your RFC with the busy reader in mind. For example, if there is a lot of technical context to give, write a summary that people can read through quickly to get a high level overview of the proposed changes, then go deeper below, or in appendices.

- It's fine to nudge people on slack if they are being slow to give feedback

Internal meetings

PostHog uses Google Meet for video communications. For large meetings, use `CMD + minus key` to zoom out and see everyone - you'll usually need to do this in All Hands.

Use video calls if you find yourself going back and forth in an issue/via email or over chat. Sometimes it is still more valuable to have a 40+ message conversation via chat as it improves transparency, is easy to refer back to, and is friendlier to newcomers getting up to speed.

1. Most scheduled meetings should have a Google Doc linked or a relevant GitHub issue. This contains an agenda, including any preparation materials.
2. Please click 'Guests can modify event' so people can update the time in the calendar instead of having to reach out via other channels. You can configure this to be checked by default under Event Settings.
3. Try to have your video on at all times because it's much more engaging for participants. Having pets, children, significant others, friends, and family visible during video chats is encouraged - please introduce them!
4. As a remote company we are always striving to have the highest fidelity, collaborative conversations. Use of a headset with a microphone, is strongly recommended - use your company card if you need.
5. Always advise participants to mute their mics if there is unnecessary background noise to ensure the speaker is able to be heard by all attendees.
6. You should take notes of the points and to-dos during the meeting. Being able to structure conclusions and follow-up actions in real time makes a video call more effective than an in-person meeting. If it is important enough to schedule a meeting, it is important enough to have taken notes.
7. We start on time and do not wait for people. People are expected to join no later than the scheduled minute of the meeting, and we don't spend time bringing latecomers up to speed.
8. It can feel rude in video calls to interrupt people. This is because the latency causes you to talk over the speaker for longer than during an in-person meeting. You should not be discouraged by this, as the questions and context provided by interruptions are valuable.
9. We end on the scheduled time. Again, it might feel rude to end a meeting, but you're actually allowing all attendees to be on time for their next meeting.
10. It is unusual to smoke or vape in an open office, and the same goes for video calls - please don't do this out of respect for others on the call.

For external meetings, the above is also helpful.

Indicating availability

1. Put your planned away time including holidays, vacation, travel time, and other leave in your own calendar.
2. Set your working hours in your Google Calendar - you can do this under *Settings* > *Working Hours*. This is helpful as we work across different timezones.

Google Calendar

We recommend you set your Google Calendar access permissions to 'Make available for PostHog - See all event details'. Consider marking the following appointments as 'Private':

1. Personal appointments
2. Particularly confidential & sensitive meetings with third-parties outside of PostHog
3. 1-1 performance or evaluation meetings
4. Meetings on organizational changes

Calendly

We use Calendly for scheduling external meetings, such as demos or product feedback calls. If you need an account, ask Simon in `#sales` to invite you to the PostHog team account.

Communication Methods

PostHog employees are frequent targets of scams and phishing. Expect all communication to occur over Slack. Phone calls, SMS, and WhatsApp are **never** used for initiating requests, approvals, or asking for sensitive info. With few exceptions, email is never used for this either.

If someone contacts you outside of Slack, treat it as **untrusted until verified**. Message them on Slack to confirm, and only continue the conversation over Slack. Other communication methods are susceptible to phishing, but our Slack instance is locked down and generally well protected from phishing and impersonation.

Best practices

James, Tim, and other execs will never ask for wire transfers, gift cards, MFA codes, or access changes over email/SMS/WhatsApp/phone. Treat such requests as phishing and report them to `#phishing-attempts`.

By email: Only trust `@posthog.com` senders. Verify via the company directory. Be cautious of look-alike domains (e.g., `posthog.co` vs `posthog.com`), unexpected attachments, and “urgent” requests.

Phone/SMS/WhatsApp: **Never** used for initiating requests, approvals, or asking for sensitive info.

If something feels off, it probably is. When in doubt, slow down and verify in Slack.